

# P2333R0

## 2021 Winter Library Evolution Poll

### Outcomes

Published Proposal, 2021-03-12

**Author:**

[Bryce Adelstein Leibach \(he/him/his\) — Library Evolution Chair](#) (NVIDIA)

**Source:**

[GitHub](#)

**Issue Tracking:**

[GitHub](#)

**Project:**

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

**Audience:**

WG21

---

## Table of Contents

### 1 Introduction

### 2 Summarized Poll Results

### 3 Selected Comments

- 3.1 Poll 1: [P0288R7] The Feature Formerly Known As `any_invocable`
- 3.2 Poll 2: [P1642R5] Freestanding [utilities], [ranges], And [iterators]
- 3.3 Poll 3: [P2216R2] `std::format` Improvements
- 3.4 Poll 4: [P2077R2] Heterogeneous Erasure Overloads For Associative Containers
- 3.5 Poll 5: [P2136R2] `invoke_r`
- 3.6 Poll 6: [P1951R0] Default Arguments For `pair`'s Forwarding Constructor
- 3.7 Poll 7: [P2231R0] Missing `constexpr` In `optional` And `variant`

3.8 Poll 8: [P0901R8] Size Feedback In operator new

3.9 Poll 9: [P1478R6] Byte-wise Atomic memcpy

## References

Informative References

## § 1. Introduction

In Winter 2021, the C++ Library Evolution group conducted a series of electronic decision polls [\[P2289R0\]](#). This paper provides the results of those polls and summarizes the results.

In total, 31 people participated in the polls. Some participants opted to not vote on some polls. Thank you to everyone who participated, and to the proposal authors for all their hard work!

## § 2. Summarized Poll Results

| Poll                                                                                                                                                                                                                                                                                                                                                                                                                                          | SF | WF | N | WA | SA | Outcome                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|----|---|----|----|-------------------------------|
| Poll 1: Modify <a href="#">[P0288R7]</a> ( <code>any_invocable</code> ) by renaming <code>any_invocable</code> to <code>move_only_function</code> (as per <a href="#">[P2265R0]</a> Renaming <code>any_invocable</code> ) and placing the facility in <code>&lt;functional&gt;</code> instead of adding a new header, and then send the revised paper to LWG for C++23, classified as an addition ( <a href="#">[P0592R4]</a> bucket 3 item). | 9  | 13 | 4 | 1  | 2  | Consensus in favor.           |
| Poll 2: Send <a href="#">[P1642R5]</a> (Freestanding [utilities], [ranges], And [iterators]) to LWG, classified as an improvement of an existing feature ( <a href="#">[P0592R4]</a> bucket 2 item).                                                                                                                                                                                                                                          | 13 | 13 | 0 | 0  | 0  | Unanimous consensus in favor. |
| Poll 3: Send <a href="#">[P2216R2]</a> ( <code>std::format</code> Improvements) to LWG for C++23, classified as an improvement of an existing feature ( <a href="#">[P0592R4]</a> bucket 2 item).                                                                                                                                                                                                                                             | 15 | 11 | 1 | 0  | 0  | Strong consensus in favor.    |
| Poll 4: Send <a href="#">[P2077R2]</a> (Heterogeneous Erasure Overloads For Associative Containers) to LWG for                                                                                                                                                                                                                                                                                                                                | 12 | 11 | 0 | 0  | 0  | Unanimous consensus in        |

|                                                                                                                                                                                                                                                                                            |    |    |   |   |   |                            |        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|----|---|---|---|----------------------------|--------|
| C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).                                                                                                                                                                                                      |    |    |   |   |   |                            | favor. |
| Poll 5: Send [P2136R2] (invoke_r) to LWG for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).                                                                                                                                                         | 11 | 12 | 1 | 0 | 0 | Strong consensus in favor. |        |
| Poll 6: Modify [P1951R0] (Default Arguments For pair's Forwarding Constructor) by rebasing the wording on the working draft, and then send the revised paper to LWG for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).                              | 14 | 9  | 2 | 0 | 0 | Strong consensus in favor. |        |
| Poll 7: Modify [P2231R0] (Missing constexpr In optional And variant) by adding bumps of the feature test macros __cpp_lib_optional and __cpp_lib_variant, and then send the revised paper to LWG for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item). | 19 | 8  | 1 | 0 | 0 | Strong consensus in favor. |        |
| Poll 8: Send [P0901R8] (Size Feedback In operator new) to CWG for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).                                                                                                                                    | 12 | 10 | 0 | 1 | 0 | Consensus in favor.        |        |
| Poll 9: Send [P1478R6] (Byte-wise Atomic memcpy) to LWG for the Concurrency TS v2, classified as an addition ([P0592R4] bucket 3 item).                                                                                                                                                    | 6  | 11 | 2 | 1 | 0 | Consensus in favor.        |        |

## § 3. Selected Comments

### § 3.1. Poll 1: [\[P0288R7\]](#) The Feature Formerly Known As `any_invocable`

The name now fits into its surrounding facilities, and gives useful hints about the properties of the function wrapper.

— Strongly Favor

Consistency between function wrapper types (using the word `function`) is valuable, as is a prefix that distinguishes the semantics of `move_only_function` from `function`.

— Strongly Favor

I like this name change. It would be annoying for the "move-only flavor of `function`" to have an entirely different name from `function`. Perhaps it was a bad idea to name `function` the way we did, but that ship has sailed, and `any_invocable` makes it worse in two different ways (`function` not in the name, and `any_*` falsely suggests that `std::any` is involved).

— Weakly Favor

The name is good enough, and the facilities are overdue.

— Weakly Favor

The facility is plainly needed; no one has found a perfect name, but the proposed name is acceptable.

— Weakly Favor

I do not much care for this rename, but it works.

— Weakly Favor

Basically, I am not sure that `move_only_function` is the best name that could be. To me it's better than `any_invocable` and that's why I am weakly in favor. But I am trying to look at that from the user perspective. As the user I have `function` and `move_only_function`. From the name I cannot get why I need `move_only_function` and when it's better than just `function`. I understand that it can work with move only callables but I believe `move_only_function` should also work with copyable callables So, basically user question is: "why do I need to use `move_only_function` with a copyable callable?". `unique_function` would probably be better name. It's consistent with `unique_ptr` and still owns the "copy" of the callable underneath (even if it has a slightly different semantics comparing with `unique_ptr`) Or we should try to find something else.

— Weakly Favor

While I'm for placing the facility in `<functional>`, I think `move_only_function` is a unconscionable name: The target need not be move only (it could be copyable or not movable at all) and the target need not be a function. If the name is construed as describing the wrapper rather than the target it's still inaccurate: `is_function_v<move_only_function<...>>` is always false.

— Strongly Against

In my experience, `any_invocable` is used more often than `function` - few callbacks and predicates need or can be copied and therefore, `any_invocable` is the more general solution, before even talking about the performances of `any_invocable` being much more palatable than that of `function`, `const correctness`, etc.

We should therefore nudge people toward `any_invocable`, alas, `move_only_function` makes the type look like an oddity that should not be used through an usually long name, which unusually refers to a specific property of the type.

Why not `move_only_const_correct_function` then?

Moving away from the `any_*` will continue to cause naming debates for future type erased types.

— Strongly Against

### § 3.2. Poll 2: [\[P1642R5\]](#) Freestanding [utilities], [ranges], And [iterators]

Anything that adds to freestanding makes it easier for many segments to use the C++ library without hacks to get around the limitations.

— Strongly Favor

Anything that is trivial to make freestanding should be, since the implementation effort is necessary in general regardless.

— Weakly Favor

Maybe the choice of a `// freestanding` comment is unconventional, but it is a direction towards a clearer definition of what's available in a freestanding environment.

— Weakly Favor

### § 3.3. Poll 3: [\[P2216R2\]](#) `std::format` Improvements

Compile time checking and compilation of format strings will increase runtime performance and catch errors earlier in the design-code-build-test cycle.

— Strongly Favor

This is major improvement to `format` both in terms of safety and binary footprint. It is a good opportunity to make the API right before major implementations ship the feature.

— Strongly Favor

Losing the ability to wrap `format` is concerning but `vformat` is probably an acceptable workaround for now. There may be a case for exposing the compile-time checking functionality to users so that they can take advantage of it in their wrappers.

— Weakly Favor

Changes like this should come sooner rather than later to minimize the divergence of practical C++20 implementations.

— Weakly Favor

### § 3.4. Poll 4: [\[P2077R2\]](#) Heterogeneous Erasure Overloads For Associative Containers

This fixes an embarrassing issue with associative containers, where the (to users) obvious code is performance-penalized.

— Strongly Favor

Since `find` became heterogeneous the need to `find` and then erase on an iterator was klunky: No reason for erase not to be able to do this directly.

— Strongly Favor

Improves consistency and symmetry of existing interface, as well as opportunities for efficiency.

— Weakly Favor

### § 3.5. Poll 5: [\[P2136R2\]](#) `invoke_r`

The fact that `is_invocable_r` and friends existed without `invoke_r` was weird and led to ad hoc implementations thereof.

— Strongly Favor

Not being able to specify the return type comes up with enough frequency that fixing it is well worth the effort.

— Weakly Favor

This improves consistency with the existing type traits and provides a capability sufficiently useful to already be employed by the standard library in multiple places.

— Weakly Favor

### § 3.6. Poll 6: [\[P1951R0\]](#) Default Arguments For `pair`'s Forwarding Constructor

This fixes something that's currently broken.

— Strongly Favor

This fixes the behavior of code that contains a silent performance bug, as well as allowing certain cases involving non-movable types.

— Strongly Favor

Eliminates a usage gotcha, so an improvement.

— Weakly Favor

### § 3.7. Poll 7: [\[P2231R0\]](#) Missing `constexpr` In `optional` And `variant`

More `constexpr` in generic library code is better than the alternative especially given the role `optional` & `variant` have as "vocabulary types."

— Strongly Favor

It's silly to not consider this a fix for C++20, given the core-language changes there, but the sooner the better.

— Strongly Favor

This is an important `constexpr` extension with good implementation experience.

— Weakly Favor

### § 3.8. Poll 8: [\[P0901R8\]](#) Size Feedback In operator new

Seems like a straightforward improvement: Hiding useful information is the number one way to have non-zero cost abstractions.

— Strongly Favor

I feel like the size was always a required result for allocation. I like this and would like it to become the primary allocation interface.

— Weakly Favor

This is a good addition to C++. My one concern is that sometimes more space is allocated for performance, and sometimes memory is so constrained on embedded systems that extra memory is not wanted unless it would have been allocated anyway. This interface does not give the caller a way to differentiate between those two requests.

— Weakly Favor

I appreciate the situation, but I'm not sure that the solution is at the right level of abstraction.

— Weakly Against

### § 3.9. Poll 9: [\[P1478R6\]](#) Byte-wise Atomic memcpy

This is a good candidate for something that should be added to the Concurrency TS, as we need the feedback to answer the open questions.

— Strongly Favor

I've seen real world use cases where this facility would've allowed the code written to be more correct.

— Strongly Favor

Being able to utilize the (weak) semantics of actual memory operations is much better than having formal undefined behavior that programmers are incentivized to ignore.

— Weakly Favor

Specialized facility but definitely standard library material

— Weakly Favor

I generally dislike the TS process, but this does seem to need much wider feedback after implementation to answer the questions raised in the paper.

— Weakly Favor

Given the current backlog, putting this feature in a TS with an uncertain future does not seem conducive to progress.

— Weakly Against

## § References

### § Informative References

#### [P0288R7]

Ryan McDougall, Matt Calabrese. [any\\_invocable](https://wg21.link/p0288r7). 3 September 2020. URL: <https://wg21.link/p0288r7>

#### [P0592R4]

Ville Voutilainen. [To boldly suggest an overall plan for C++23](https://wg21.link/p0592r4). 25 November 2019. URL: <https://wg21.link/p0592r4>

#### [P0901R8]

Chris Kennelly, Andrew Hunter. [Size feedback in operator new](https://wg21.link/p0901r8). 15 December 2020. URL: <https://wg21.link/p0901r8>

#### [P1478R6]

Hans Boehm. [Byte-wise atomic memcpy](https://wg21.link/p1478r6). 14 December 2020. URL: <https://wg21.link/p1478r6>

#### [P1642R5]

Ben Craig. [Freestanding Library: Easy \[utilities\], \[ranges\], and \[iterators\]](https://wg21.link/p1642r5). 10 December 2020. URL: <https://wg21.link/p1642r5>

**[P1951R0]**

Logan R. Smith. [Default Arguments for pair's Forwarding Constructor](https://wg21.link/p1951r0). 17 November 2019. URL: <https://wg21.link/p1951r0>

**[P2077R2]**

Konstantin Boyarinov, Sergey Vinogradov; Ruslan Arutyunyan. [Heterogeneous erasure overloads for associative containers](https://wg21.link/p2077r2). 15 December 2020. URL: <https://wg21.link/p2077r2>

**[P2136R2]**

Zhihao Yuan. [invoke\\_r](https://wg21.link/p2136r2). 6 December 2020. URL: <https://wg21.link/p2136r2>

**[P2216R2]**

Victor Zverovich. [std::format improvements](https://wg21.link/p2216r2). 15 January 2021. URL: <https://wg21.link/p2216r2>

**[P2231R0]**

Barry Revzin. [Add further constexpr support for optional/variant](https://wg21.link/p2231r0). 14 October 2020. URL: <https://wg21.link/p2231r0>

**[P2265R0]**

Kevlin Henney. [Renaming any\\_invocable](https://wg21.link/p2265r0). 6 December 2020. URL: <https://wg21.link/p2265r0>

**[P2289R0]**

Bryce Adelstein Leibach. [2021 Winter Library Evolution Polls](https://wg21.link/p2289r0). 1 February 2021. URL: <https://wg21.link/p2289r0>